前置前置

这里是存放一些大模型的基础知识，通过了解这些知识来完成自己的Claude吧

关键词

- prompt：输入给大模型的内容

> prompt engineering:研究怎么写prompt才能让模型输出更好的结果

- system promp：在对话开始前，给模型设定一个"角色"和"行为范式"，模型在整个对话过程中都会遵守这个设定
- token：大模型处理文本的最小单位
- 上下文/上下文窗口：跟模型聊天，所有的输入——System Prompt、对话历史、RAG 检索到的文档、工具返回的结果——都装在一个"窗口"里，这个窗口就叫上下文。

> 这个窗口有大小限制，称为上下文窗口
> 当窗口满了，最早的对话内容会被挤出去，让模型忘记之前的对话
> 窗口并非越大越好，窗口越大，成本与延迟更高，研究表明模型在超长上下文中可能出现"Lost in the Middle"现象——中间的内容模型反而注意不到

- 幻觉：大模型会编造不存在的东西，这是因为大模型的工作方式本质上是在预测下一个最可能出现的token，而不是在检索事实

> function calling:让模型去调用真实的接口查数据，而不是自己编答案。规定模型在对应情况调用对应的工具
> RAG：检索增强生成，把真实的文档检索出来放入上下文，让模型基于事实回答
> Structured output：约束输出格式，减少模型自由发挥的空间。基于结构化数据输出，例如json、表格或者固定字段

- RAG：检索增强生成。离线阶段：将文档切成片段，转成向量，存进数据库。在线阶段：用户提问时，检索最相关的片段，加入上下文，模型基于这些片段回答
- Embedding：把文本翻译成一组数字(向量)，语义越相近的文本，向量距离越近。RAG检索不是搜关键词，而是算向量距离，找到语义最相近的文档片段

agent

agent循环：

1. 接收用户任务
2. 规划事情分几步
3. 执行第一步：调用工具或检索资料或直接回答
4. 观察结果，判断是否继续
5. 如果没完成，回到步骤2
6. 任务完成，输出最终结果
   Agent 的本质 = 思维链（规划能力）+ Function Calling（执行能力）+ 循环（迭代能力）

MCP

MCP（Model Context Protocol） 就是 AI 领域的"USB-C"：一个标准化的协议，让任何模型都能用统一的方式连接外部工具和数据源

- 工具开发者只需要实现一次 MCP 协议，所有支持 MCP 的模型都能用
- 应用开发者不需要为每个工具写适配代码
- 换模型不需要改工具对接逻辑

skill

Skill 把 Agent 的某个能力封装成一个可复用的"技能包"

- 不同 Agent 之间可以共享能力
- 新 Agent 不用从零开发，组合现有 Skill 就行
- 能力可以独立升级，不影响其他部分